

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
SIMATS ENGINEERING
COMPUTER SCIENCE AND ENGINEERING
CSA17 - ARTIFICIAL INTELLIGENCE LABORATORY RECORD

Programs 11 - 15

Program 11: Tic Tac Toe Game

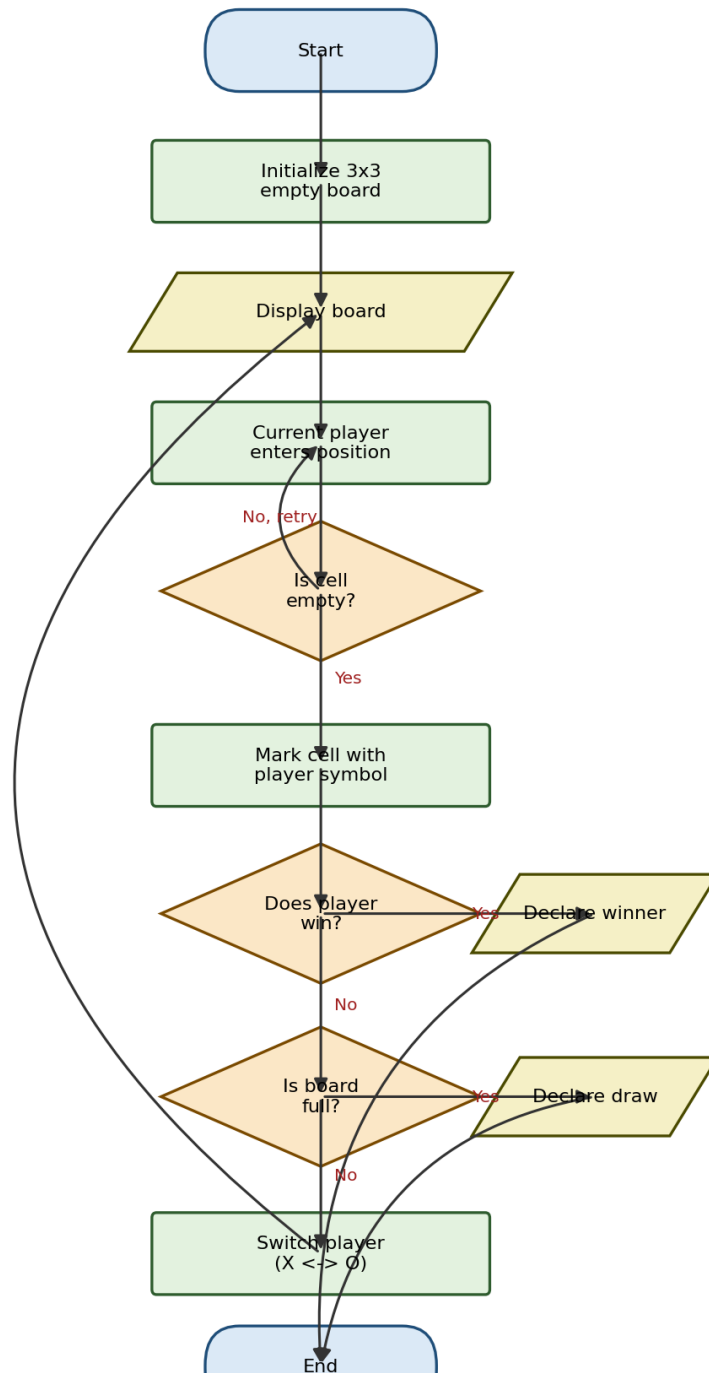
Aim

To write a Python program to implement the two-player Tic-Tac-Toe game using a 3x3 board with console-based input and output.

Algorithm

1. Start.
2. Initialize a list of size 9 with empty spaces to represent the 3x3 board. Set the current player to 'X'.
3. Display the current state of the board.
4. Prompt the current player to enter a position (1-9).
5. If the chosen position is already occupied, display an error message and repeat step 4.
6. Otherwise, mark the chosen position with the current player's symbol and display the updated board.
7. Check all 8 winning combinations (3 rows, 3 columns, 2 diagonals) to see if the current player has won.
8. If the current player has won, display the winner and stop.
9. If the board is full and no one has won, declare the game a draw and stop.
10. Otherwise, switch to the other player and repeat from step 3.
11. Stop.

Flowchart

Flowchart: Tic Tac Toe Game

Program

```

"""
Program 11: Tic Tac Toe Game
Two-player console based Tic Tac Toe using Python
"""

def print_board(board):
    print()
    for i in range(3):
        row = " | ".join(board[i*3:i*3+3])
        print(" " + row)
        if i < 2:
            print("---+---+---")
    print()

def check_winner(board, player):
    win_positions = [
        [0, 1, 2], [3, 4, 5], [6, 7, 8], # rows
        [0, 3, 6], [1, 4, 7], [2, 5, 8], # columns
        [0, 4, 8], [2, 4, 6]             # diagonals
    ]
    for pos in win_positions:
        if all(board[i] == player for i in pos):
            return True
    return False

def is_full(board):
    return all(cell != ' ' for cell in board)

def play_game(moves=None):
    board = [' '] * 9
    current_player = 'X'
    move_index = 0

    print("Welcome to Tic Tac Toe!")
    print_board(board)

```

```

while True:
    if moves is not None:
        position = moves[move_index]
        move_index += 1
        print(f'Player {current_player} chooses position {position + 1}')
    else:
        position = int(input(f'Player {current_player}, enter position (1-9): ')) - 1

    if board[position] != ' ':
        print("Position already taken! Try again.")
        continue

    board[position] = current_player
    print_board(board)

    if check_winner(board, current_player):
        print(f'Player {current_player} wins!')
        break

    if is_full(board):
        print("It's a draw!")
        break

    current_player = 'O' if current_player == 'X' else 'X'

if __name__ == "__main__":
    # Sample predefined moves to demonstrate a complete game (X wins)
    sample_moves = [0, 1, 4, 2, 8, 6, 7]
    play_game(sample_moves)

```

Sample Input

A predefined sequence of moves is used to demonstrate a complete game:

Player X -> position 1, Player O -> position 2, Player X -> position 5,

Player O -> position 3, Player X -> position 9.

Sample Output

Welcome to Tic Tac Toe!

```

| |
---+---+---
| |
---+---+---
| |

```

Player X chooses position 1

```

X | |
---+---+---
| |
---+---+---
| |

```

Player O chooses position 2

```

X | O |
---+---+---
| |
---+---+---
...

```

The board is printed after every move. The program detects that Player X has completed the diagonal (1, 5, 9) and declares "Player X wins!"

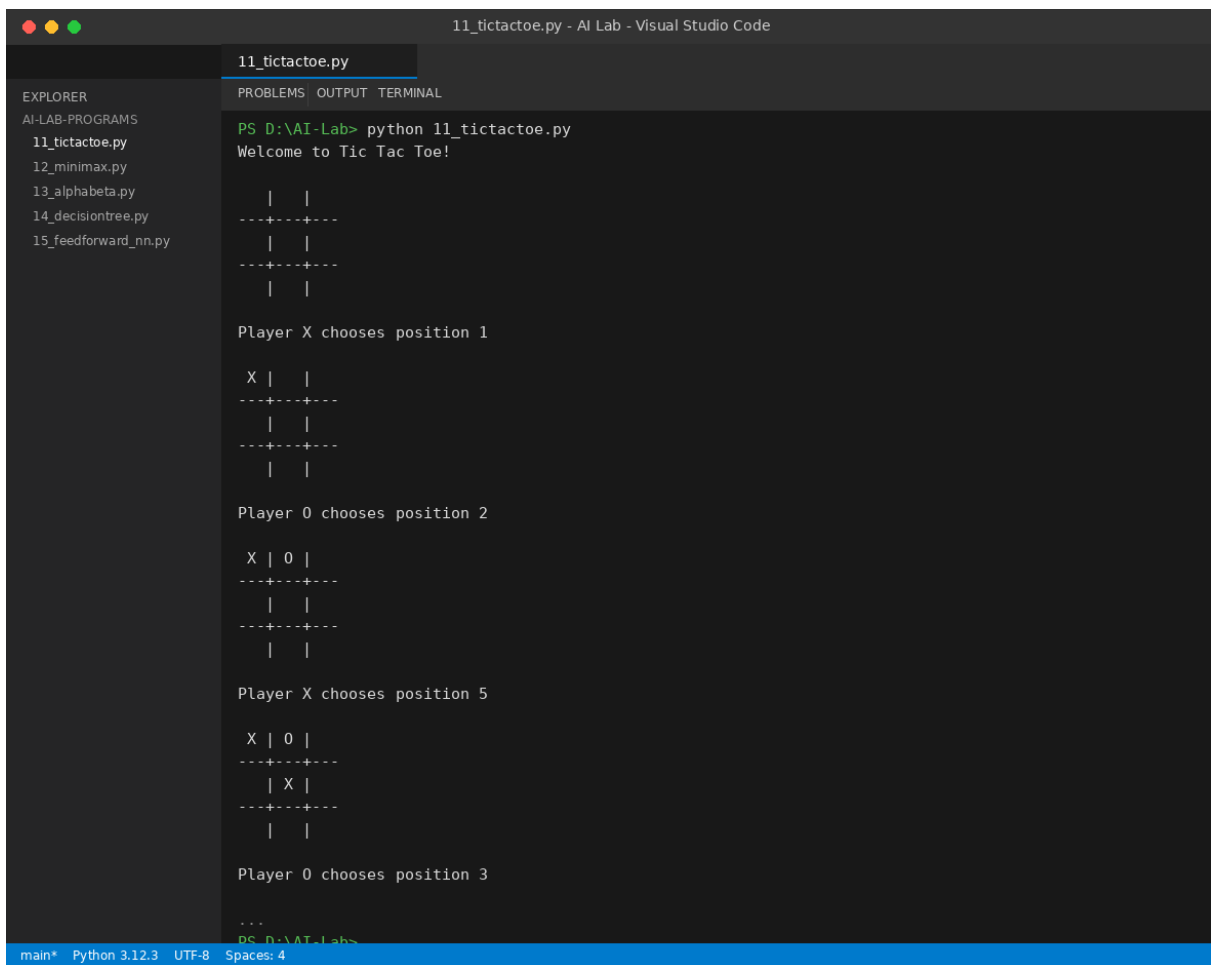
Sample Screenshot (Program running in VS Code)

```

1  """
2  Program 11: Tic Tac Toe Game
3  Two-player console based Tic Tac Toe using Python
4  """
5
6  def print_board(board):
7      print()
8      for i in range(3):
9          row = " | ".join(board[i*3:i*3+3])
10         print(" " + row)
11         if i < 2:
12             print("----+---")
13     print()
14
15 def check_winner(board, player):
16     win_positions = [
17         [0, 1, 2], [3, 4, 5], [6, 7, 8], # rows
18         [0, 3, 6], [1, 4, 7], [2, 5, 8], # columns
19         [0, 4, 8], [2, 4, 6] # diagonals
20     ]
21     for pos in win_positions:
22         if all(board[i] == player for i in pos):
23             return True
24     return False
25
26 def is_full(board):
27     return all(cell != ' ' for cell in board)
28
29 def play_game(moves=None):
30     board = [' '] * 9
31     current_player = 'X'
32     move_index = 0
33
34     print("Welcome to Tic Tac Toe!")
35     print_board(board)
36
37     while True:
38         if moves is not None:
39             position = moves[move_index]
40             move_index += 1
41             print(f"Player {current_player} chooses position {position + 1}")
42         else:
43             position = int(input(f"Player {current_player}, enter position (1-9): ")) - 1
44
45             if board[position] != ' ':
46                 print("Position already taken! Try again.")
47                 continue
48
49             board[position] = current_player
50             print_board(board)
51
52             if check_winner(board, current_player):
53                 print(f"Player {current_player} wins!")
54                 break
55
56             if is_full(board):
57                 print("It's a draw!")
58                 break
59
60             current_player = 'O' if current_player == 'X' else 'X'
61
62
63 if __name__ == "__main__":
64     # Sample predefined moves to demonstrate a complete game (X wins)
65     sample_moves = [0, 1, 4, 2, 8, 6, 7]
66     play_game(sample_moves)

```

Fig 11.1 - Editor view of 11_tictactoe.py in Visual Studio Code



```
11_tictactoe.py
PROBLEMS OUTPUT TERMINAL

PS D:\AI-Lab> python 11_tictactoe.py
Welcome to Tic Tac Toe!

  | |
  +---+
  | |
  +---+
  | |

Player X chooses position 1

X | |
  +---+
  | |
  +---+
  | |

Player O chooses position 2

X | O |
  +---+
  | |
  +---+
  | |

Player X chooses position 5

X | O |
  +---+
  | X |
  +---+
  | |

Player O chooses position 3

...
PS D:\AI-Lab>
```

Fig 11.2 - Integrated terminal showing the program output

Result

Thus the Python program for "Tic Tac Toe Game" was executed successfully and the output was verified.

Program 12: Minimax Algorithm for Gaming

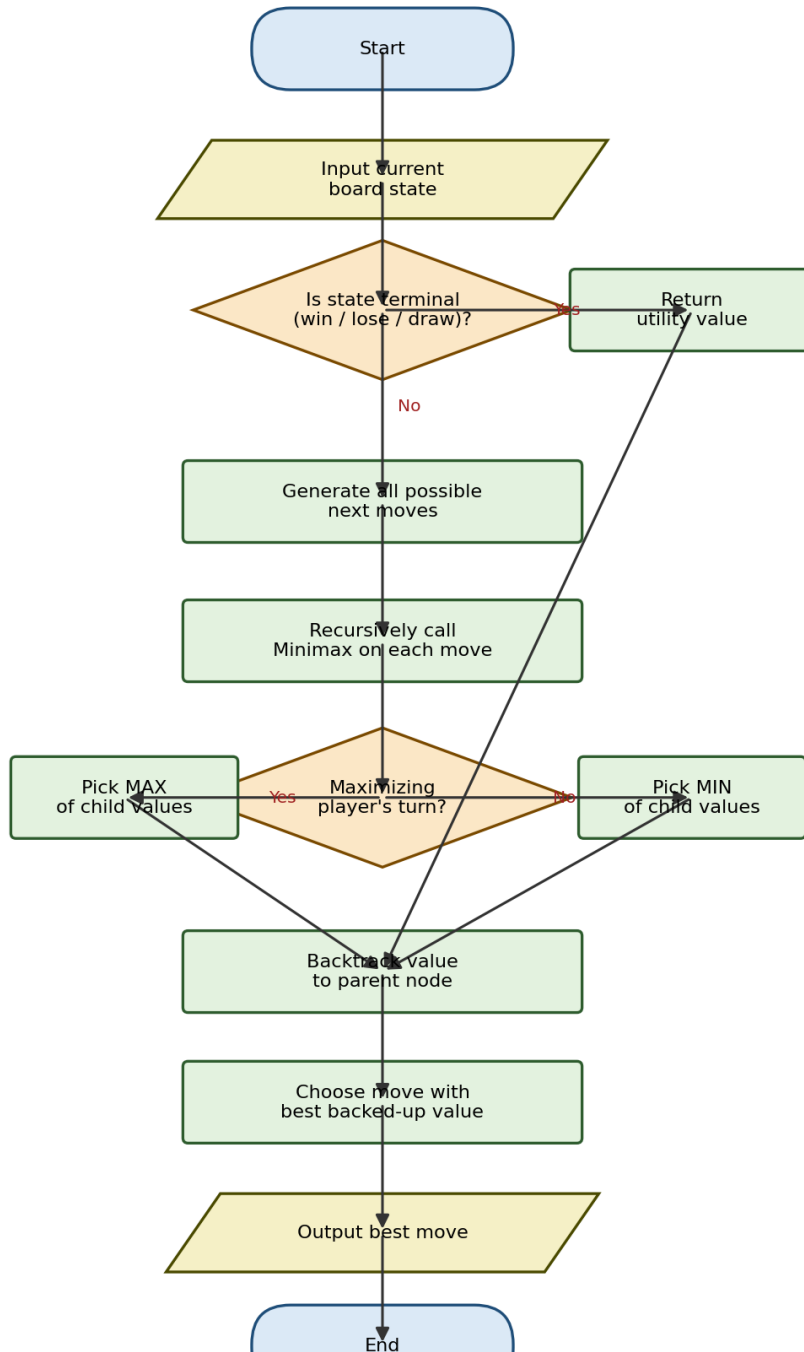
Aim

To write a Python program to implement the Minimax algorithm so that the computer plays Tic-Tac-Toe optimally (never loses).

Algorithm

1. Start.
2. Represent the current Tic-Tac-Toe board as the game state.
3. Define a recursive function Minimax(board, depth, isMaximizing).
4. If the board is a terminal state (a win, a loss, or a draw), return its utility value: (10 - depth) if the computer (O) wins, (depth - 10) if the opponent (X) wins, and 0 for a draw.
5. If it is the maximizing player's (computer's) turn, try placing 'O' in every empty cell, recursively call Minimax for each resulting board, and return the maximum of the returned scores.
6. If it is the minimizing player's (opponent's) turn, try placing 'X' in every empty cell, recursively call Minimax for each resulting board, and return the minimum of the returned scores.
7. At the top level, evaluate the Minimax score for every currently empty cell and choose the move that produces the maximum score.
8. Place the computer's symbol ('O') in the chosen cell.
9. Display the resulting board.
10. Stop.

Flowchart

Flowchart: Minimax Algorithm for Gaming

Program

```

"""
Program 12: Minimax Algorithm for Gaming
Applying Minimax to Tic Tac Toe so the computer (O) never loses
"""

def print_board(board):
    print()
    for i in range(3):
        row = " | ".join(board[i*3:i*3+3])
        print(" " + row)
        if i < 2:
            print("---+---+---")
    print()

def check_winner(board, player):
    win_positions = [
        [0, 1, 2], [3, 4, 5], [6, 7, 8],
        [0, 3, 6], [1, 4, 7], [2, 5, 8],
        [0, 4, 8], [2, 4, 6]
    ]
    return any(all(board[i] == player for i in pos) for pos in win_positions)

def is_full(board):
    return all(cell != ' ' for cell in board)

def minimax(board, depth, is_maximizing):
    if check_winner(board, 'O'):
        return 10 - depth
    if check_winner(board, 'X'):
        return depth - 10
    if is_full(board):
        return 0

    if is_maximizing:
        best_score = float('-inf')
        for i in range(9):

```

```

        if board[i] == ' ':
            board[i] = 'O'
            score = minimax(board, depth + 1, False)
            board[i] = ' '
            best_score = max(best_score, score)
    return best_score
else:
    best_score = float('inf')
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = minimax(board, depth + 1, True)
            board[i] = ' '
            best_score = min(best_score, score)
    return best_score

def best_move(board):
    best_score = float('-inf')
    move = -1
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = minimax(board, 0, False)
            board[i] = ' '
            if score > best_score:
                best_score = score
                move = i
    return move

if __name__ == "__main__":
    # X has already played two moves; it is O's (computer's) turn to move
    board = ['X', ' ', ' ',
             ' ', 'X', ' ',
             ' ', ' ', ' ']

    print("Current board:")
    print_board(board)

```

```

move = best_move(board)
board[move] = 'O'

print(f'Computer (O) plays best move at position {move + 1} to block X:')
print_board(board)

```

Sample Input

A partially filled board is given as input:

Row1: X | _ | _ Row2: _ | X | _ Row3: _ | _ | _

(It is the computer's (O's) turn to move.)

Sample Output

Current board:

```

X | | 
---+---+---
| X | 
---+---+---
| | 

```

Computer (O) plays best move at position 9 to block X:

```

X | | 
---+---+---
| X | 
---+---+---
| | O

```

Minimax evaluates every empty cell and finds that placing 'O' at position 9 gives the best (safest) score, so the computer plays at position 9.

Sample Screenshot (Program running in VS Code)



```

12_minimax.py

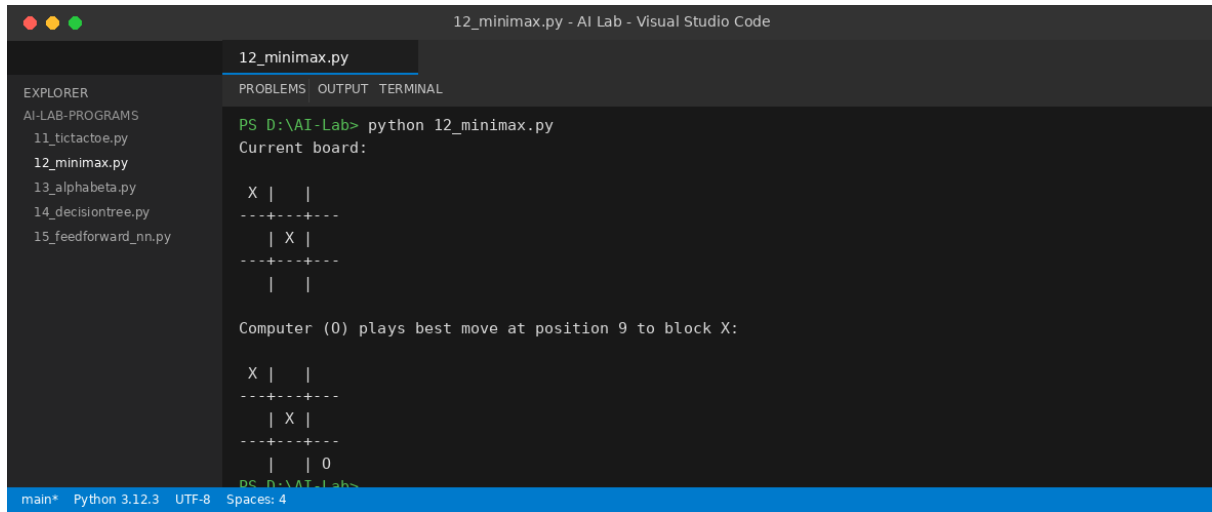
EXPLORER
AI-LAB-PROGRAMS
  11_tictactoe.py
  12_minimax.py
  13_alphabeta.py
  14_decisiontree.py
  15_feedforward_nn.py

1  """
2  Program 12: Minimax Algorithm for Gaming
3  Applying Minimax to Tic Tac Toe so the computer (0) never loses
4  """
5
6  def print_board(board):
7      print()
8      for i in range(3):
9          row = " | ".join(board[i*3:i*3+3])
10         print(" " + row)
11         if i < 2:
12             print("-----")
13     print()
14
15  def check_winner(board, player):
16      win_positions = [
17          [0, 1, 2], [3, 4, 5], [6, 7, 8],
18          [0, 3, 6], [1, 4, 7], [2, 5, 8],
19          [0, 4, 8], [2, 4, 6]
20      ]
21      return any(all(board[i] == player for i in pos) for pos in win_positions)
22
23  def is_full(board):
24      return all(cell != ' ' for cell in board)
25
26  def minimax(board, depth, is_maximizing):
27      if check_winner(board, 'O'):
28          return 10 - depth
29      if check_winner(board, 'X'):
30          return depth - 10
31      if is_full(board):
32          return 0
33
34      if is_maximizing:
35          best_score = float('-inf')
36          for i in range(9):
37              if board[i] == ' ':
38                  board[i] = 'O'
39                  score = minimax(board, depth + 1, False)
40                  board[i] = ' '
41                  best_score = max(best_score, score)
42          return best_score
43      else:
44          best_score = float('inf')
45          for i in range(9):
46              if board[i] == ' ':
47                  board[i] = 'X'
48                  score = minimax(board, depth + 1, True)
49                  board[i] = ' '
50                  best_score = min(best_score, score)
51          return best_score
52
53  def best_move(board):
54      best_score = float('-inf')
55      move = -1
56      for i in range(9):
57          if board[i] == ' ':
58              board[i] = 'O'
59              score = minimax(board, 0, False)
60              board[i] = ' '
61              if score > best_score:
62                  best_score = score
63                  move = i
64      return move
65
66  if __name__ == "__main__":
67      # X has already played two moves; it is O's (computer's) turn to move
68      board = ['X', ' ', ' ',
69              ' ', 'X', ' ',
70              ' ', ' ', ' ']
71
72      print("Current board:")
73      print_board(board)
74
75      move = best_move(board)
76      board[move] = 'O'
77
78      print(f"Computer (0) plays best move at position {move + 1} to block X:")
79      print_board(board)

```

main* Python 3.12.3 UTF-8 Spaces: 4 Ln 1, Col 1

Fig 12.1 - Editor view of 12_minimax.py in Visual Studio Code



```
12_minimax.py
PROBLEMS OUTPUT TERMINAL
PS D:\AI-Lab> python 12_minimax.py
Current board:

X | |
---+---
| X |
---+---
| |

Computer (0) plays best move at position 9 to block X:

X | |
---+---
| X |
---+---
| | 0
PS D:\AI-Lab>
```

Fig 12.2 - Integrated terminal showing the program output

Result

Thus the Python program for "Minimax Algorithm for Gaming" was executed successfully and the output was verified.

Program 13: Alpha & Beta Pruning Algorithm for Gaming

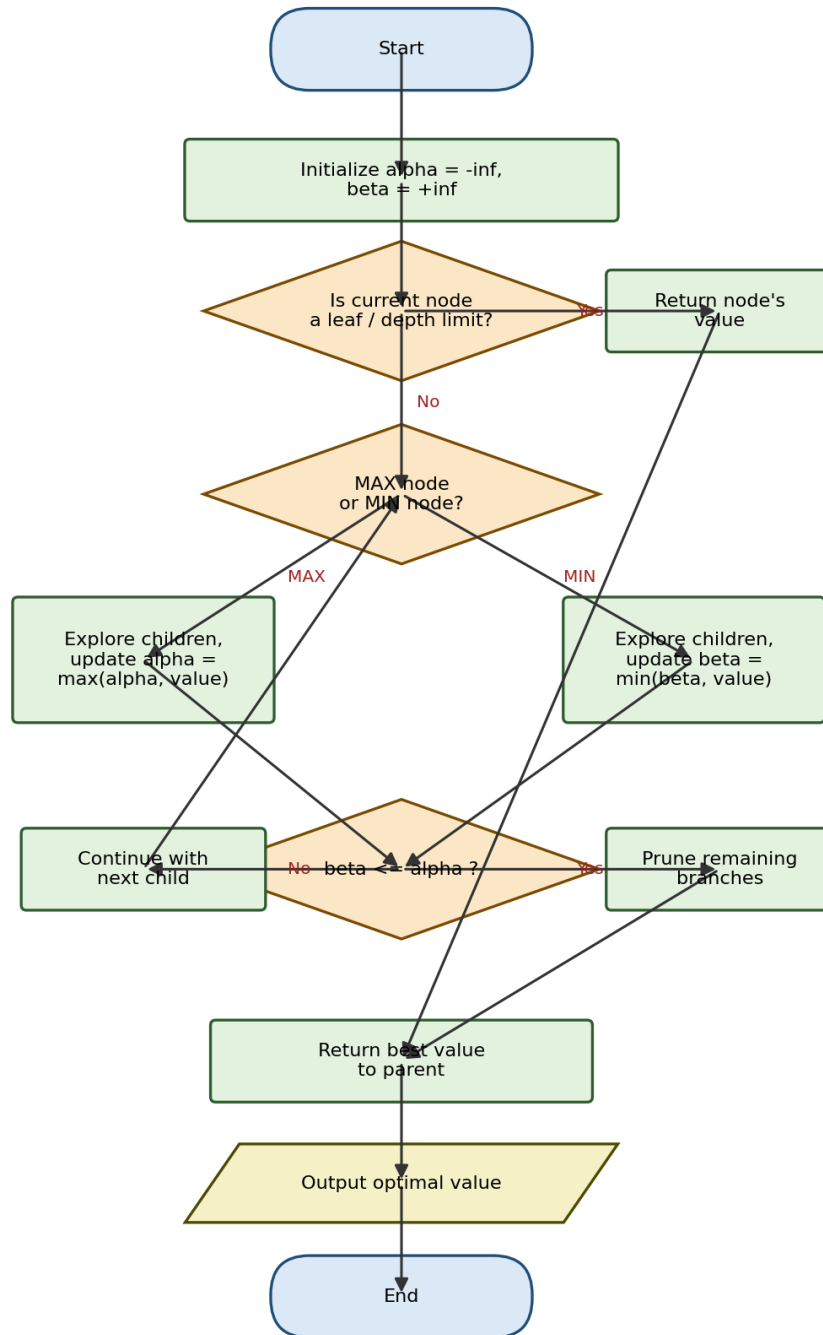
Aim

To write a Python program to implement the Alpha-Beta pruning algorithm that optimizes the Minimax search by pruning branches of the game tree that cannot influence the final decision.

Algorithm

1. Start.
2. Represent the game tree with leaf nodes holding fixed utility values.
3. Initialize $\alpha = -\text{infinity}$ and $\beta = +\text{infinity}$ for the root call.
4. If the current node is a leaf node or the maximum search depth is reached, return its value.
5. If the current node is a MAX node: initialize $\text{best} = -\text{infinity}$; for each child, recursively evaluate its value, set $\text{best} = \max(\text{best}, \text{value})$ and $\alpha = \max(\alpha, \text{best})$; if $\beta \leq \alpha$, stop exploring (prune) the remaining children.
6. If the current node is a MIN node: initialize $\text{best} = +\text{infinity}$; for each child, recursively evaluate its value, set $\text{best} = \min(\text{best}, \text{value})$ and $\beta = \min(\beta, \text{best})$; if $\beta \leq \alpha$, stop exploring (prune) the remaining children.
7. Return the best value found to the parent node.
8. Repeat the process up the tree until the value at the root is obtained.
9. Display the optimal value and the number of leaf nodes actually evaluated (i.e., not pruned).
10. Stop.

Flowchart

Flowchart: Alpha-Beta Pruning Algorithm

Program

```

"""
Program 13: Alpha-Beta Pruning Algorithm for Gaming
Demonstrated on a sample game tree
"""

# Sample game tree represented as leaf node values (depth = 3, branching factor = 2)
leaves = [3, 5, 6, 9, 1, 2, 0, -1]
node_count = 0 # counts how many leaf nodes were actually evaluated

def alphabeta(depth, node_index, is_maximizing, values, alpha, beta, max_depth):
    global node_count
    if depth == max_depth:
        node_count += 1
        return values[node_index]

    if is_maximizing:
        best = float('-inf')
        for i in range(2):
            val = alphabeta(depth + 1, node_index * 2 + i, False, values, alpha, beta,
max_depth)
            best = max(best, val)
            alpha = max(alpha, best)
            if beta <= alpha:
                break # beta cut-off (prune)
        return best
    else:
        best = float('inf')
        for i in range(2):
            val = alphabeta(depth + 1, node_index * 2 + i, True, values, alpha, beta,
max_depth)
            best = min(best, val)
            beta = min(beta, best)
            if beta <= alpha:
                break # alpha cut-off (prune)
        return best

```

```
if __name__ == "__main__":  
    max_depth = 3  
    print("Leaf node values:", leaves)  
    optimal_value = alphabeta(0, 0, True, leaves, float('-inf'), float('inf'), max_depth)  
    print("The optimal value obtained by the MAX player is:", optimal_value)  
    print("Number of leaf nodes evaluated (after pruning):", node_count, "out of",  
          len(leaves))
```

Sample Input

A sample game tree of depth 3 with 8 leaf nodes is given:

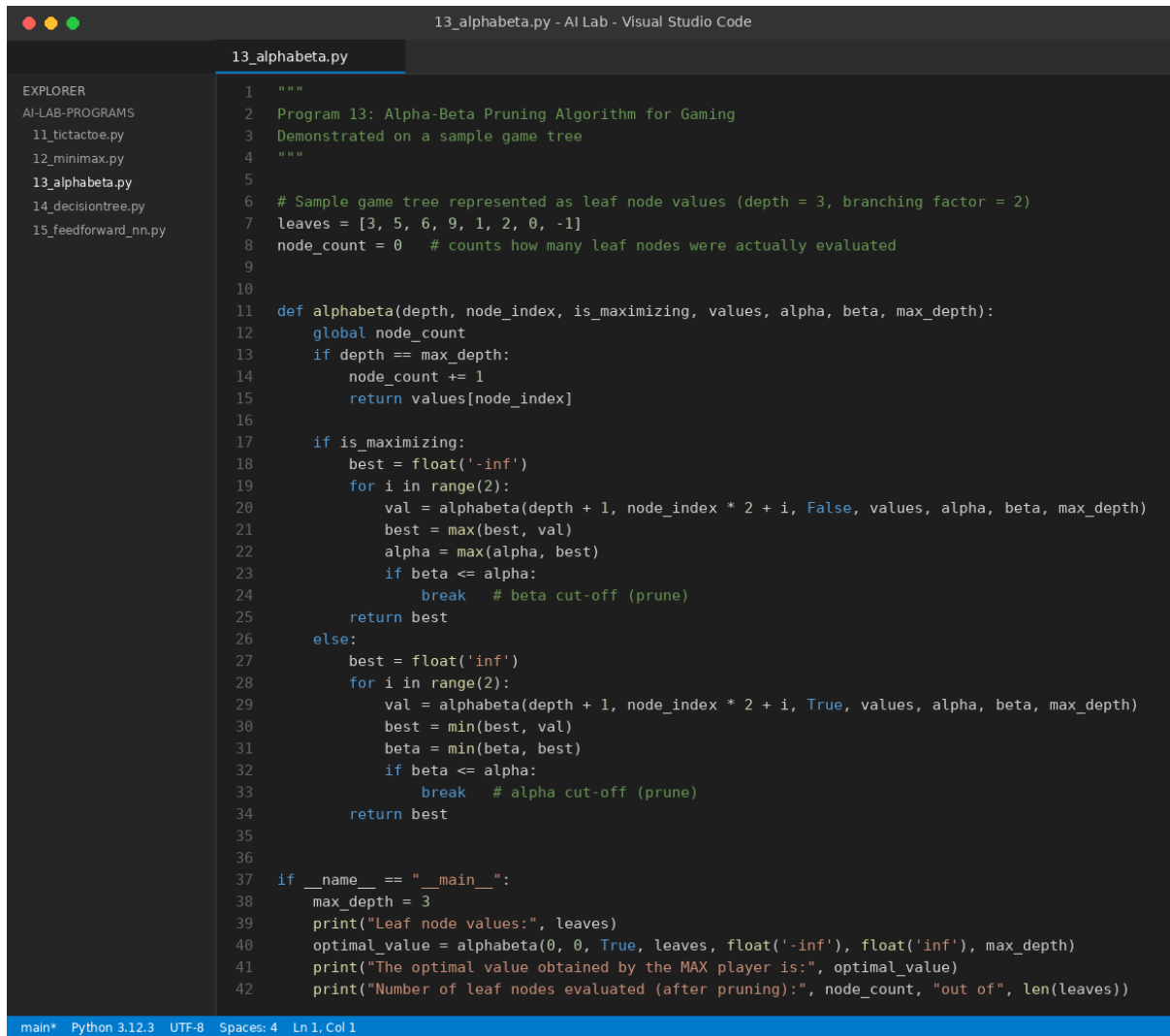
Leaf values = [3, 5, 6, 9, 1, 2, 0, -1]

Sample Output

```
Leaf node values: [3, 5, 6, 9, 1, 2, 0, -1]  
The optimal value obtained by the MAX player is: 5  
Number of leaf nodes evaluated (after pruning): 5 out of 8
```

The algorithm returns the optimal value 5 for the MAX player at the root, and it evaluates only 5 out of the 8 leaf nodes -- the remaining 3 are pruned because they cannot change the final result.

Sample Screenshot (Program running in VS Code)

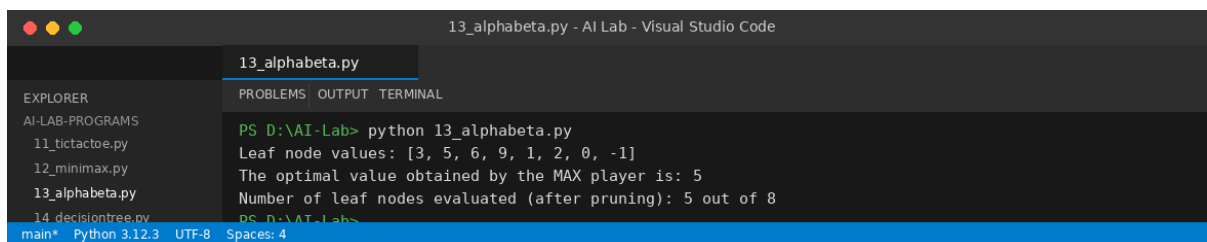


```

1  """
2  Program 13: Alpha-Beta Pruning Algorithm for Gaming
3  Demonstrated on a sample game tree
4  """
5
6  # Sample game tree represented as leaf node values (depth = 3, branching factor = 2)
7  leaves = [3, 5, 6, 9, 1, 2, 0, -1]
8  node_count = 0 # counts how many leaf nodes were actually evaluated
9
10
11 def alphabeta(depth, node_index, is_maximizing, values, alpha, beta, max_depth):
12     global node_count
13     if depth == max_depth:
14         node_count += 1
15         return values[node_index]
16
17     if is_maximizing:
18         best = float('-inf')
19         for i in range(2):
20             val = alphabeta(depth + 1, node_index * 2 + i, False, values, alpha, beta, max_depth)
21             best = max(best, val)
22             alpha = max(alpha, best)
23             if beta <= alpha:
24                 break # beta cut-off (prune)
25         return best
26     else:
27         best = float('inf')
28         for i in range(2):
29             val = alphabeta(depth + 1, node_index * 2 + i, True, values, alpha, beta, max_depth)
30             best = min(best, val)
31             beta = min(beta, best)
32             if beta <= alpha:
33                 break # alpha cut-off (prune)
34         return best
35
36
37 if __name__ == "__main__":
38     max_depth = 3
39     print("Leaf node values:", leaves)
40     optimal_value = alphabeta(0, 0, True, leaves, float('-inf'), float('inf'), max_depth)
41     print("The optimal value obtained by the MAX player is:", optimal_value)
42     print("Number of leaf nodes evaluated (after pruning):", node_count, "out of", len(leaves))

```

Fig 13.1 - Editor view of 13_alpha_beta.py in Visual Studio Code



```

PS D:\AI-Lab> python 13_alpha_beta.py
Leaf node values: [3, 5, 6, 9, 1, 2, 0, -1]
The optimal value obtained by the MAX player is: 5
Number of leaf nodes evaluated (after pruning): 5 out of 8
PS D:\AI-Lab>

```

Fig 13.2 - Integrated terminal showing the program output

Result

Thus the Python program for "Alpha & Beta Pruning Algorithm for Gaming" was executed successfully and the output was verified.

Program 14: Decision Tree

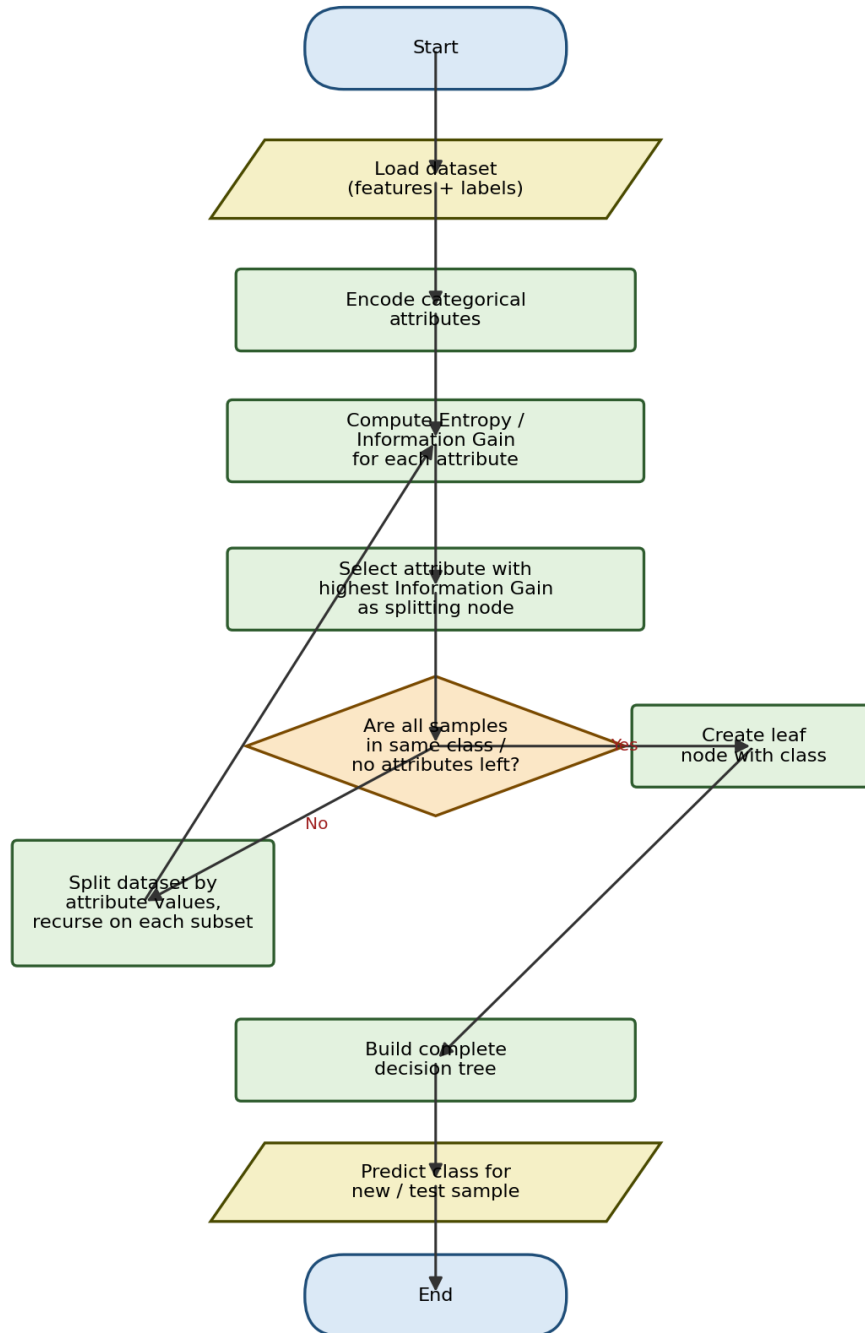
Aim

To write a Python program to construct a Decision Tree classifier for the "Play Tennis" dataset using the Entropy / Information Gain criterion and to predict the class of a new sample.

Algorithm

1. Start.
2. Load the dataset containing the attributes Outlook, Temperature, Humidity, Wind, and the class label PlayTennis.
3. Encode all categorical attributes into numeric values using Label Encoding.
4. Separate the dataset into a feature matrix X and a target vector y .
5. Compute the Entropy of the target attribute for the given set of examples.
6. For each attribute, compute the Information Gain obtained by splitting the dataset on that attribute.
7. Select the attribute with the highest Information Gain as the decision (splitting) node.
8. Partition the dataset into subsets according to the values of the selected attribute, and repeat steps 5-7 recursively for each subset.
9. Stop the recursion and create a leaf node when all samples in a subset belong to the same class, or when no attributes remain.
10. Use the trained Decision Tree Classifier to predict the class of a new / unseen sample.
11. Display the tree structure (rules), the predictions, and the accuracy.
12. Stop.

Flowchart

Flowchart: Decision Tree Construction

Program

```

"""
Program 14: Decision Tree
Build a Decision Tree classifier on the Play Tennis dataset
"""

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# Sample dataset: Play Tennis
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast',
               'Sunny', 'Sunny', 'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool',
                   'Mild', 'Cool', 'Mild', 'Mild', 'Mild', 'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal',
                'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong',
            'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Strong'],
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes',
                  'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
}

df = pd.DataFrame(data)
print("Dataset:")
print(df)

# Encode categorical columns
le = LabelEncoder()
df_encoded = df.apply(le.fit_transform)

X = df_encoded.drop('PlayTennis', axis=1)
y = df_encoded['PlayTennis']

```

```

model = DecisionTreeClassifier(criterion='entropy', random_state=1)
model.fit(X, y)

y_pred = model.predict(X)

print("\nDecision Tree Rules:")
print(export_text(model, feature_names=list(X.columns)))

print("Predicted values:", list(y_pred))
print("Actual values: ", list(y))
print("Training Accuracy:", accuracy_score(y, y_pred))

# Predict for a new sample: Outlook=Sunny, Temperature=Cool, Humidity=High,
Wind=Strong
sample = pd.DataFrame([[2, 1, 0, 1]], columns=X.columns)
prediction = model.predict(sample)
result = le.inverse_transform(prediction) if False else ('Yes' if prediction[0] == 1 else
'No')
print("\nPrediction for new sample (Sunny, Cool, High, Strong):", result)

```

Sample Input

A 14-row "Play Tennis" training dataset (Outlook, Temperature, Humidity, Wind, PlayTennis) is used for training.

New sample to classify: Outlook = Sunny, Temperature = Cool, Humidity = High, Wind = Strong.

Sample Output

Dataset:

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No

8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

Decision Tree Rules:

|--- Outlook <= 0.50

| |--- class: 1

|--- Outlook > 0.50

| |--- Humidity <= 0.50

...

The program prints the decision-tree rules (splitting first on Outlook, then Wind/Humidity), reports a training accuracy of 1.0, and predicts "No" for the new sample.

Sample Screenshot (Program running in VS Code)

```

1  """
2  Program 14: Decision Tree
3  Build a Decision Tree classifier on the Play Tennis dataset
4  """
5
6  import pandas as pd
7  from sklearn.preprocessing import LabelEncoder
8  from sklearn.tree import DecisionTreeClassifier, export_text
9  from sklearn.model_selection import train_test_split
10 from sklearn.metrics import accuracy_score
11
12 # Sample dataset: Play Tennis
13 data = {
14     'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast',
15               'Sunny', 'Sunny', 'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
16     'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool',
17                   'Mild', 'Cool', 'Mild', 'Mild', 'Mild', 'Hot', 'Mild'],
18     'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal',
19                'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal', 'High'],
20     'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong',
21             'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Strong'],
22     'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes',
23                  'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']
24 }
25
26 df = pd.DataFrame(data)
27 print("Dataset:")
28 print(df)
29
30 # Encode categorical columns
31 le = LabelEncoder()
32 df_encoded = df.apply(le.fit_transform)
33
34 X = df_encoded.drop('PlayTennis', axis=1)
35 y = df_encoded['PlayTennis']
36
37 model = DecisionTreeClassifier(criterion='entropy', random_state=1)
38 model.fit(X, y)
39
40 y_pred = model.predict(X)
41
42 print("\nDecision Tree Rules:")
43 print(export_text(model, feature_names=list(X.columns)))
44
45 print("Predicted values:", list(y_pred))
46 print("Actual values: ", list(y))
47 print("Training Accuracy:", accuracy_score(y, y_pred))
48
49 # Predict for a new sample: Outlook=Sunny, Temperature=Cool, Humidity=High, Wind=Strong
50 sample = pd.DataFrame([[2, 1, 0, 1]], columns=X.columns)
51 prediction = model.predict(sample)
52 result = le.inverse_transform(prediction) if False else ('Yes' if prediction[0] == 1 else 'No')
53 print("\nPrediction for new sample (Sunny, Cool, High, Strong):", result)

```

main* Python 3.12.3 UTF-8 Spaces: 4 Ln 1, Col 1

Fig 14.1 - Editor view of 14_decisiontree.py in Visual Studio Code

Result

Page 27

Program 15: Feed Forward Neural Network

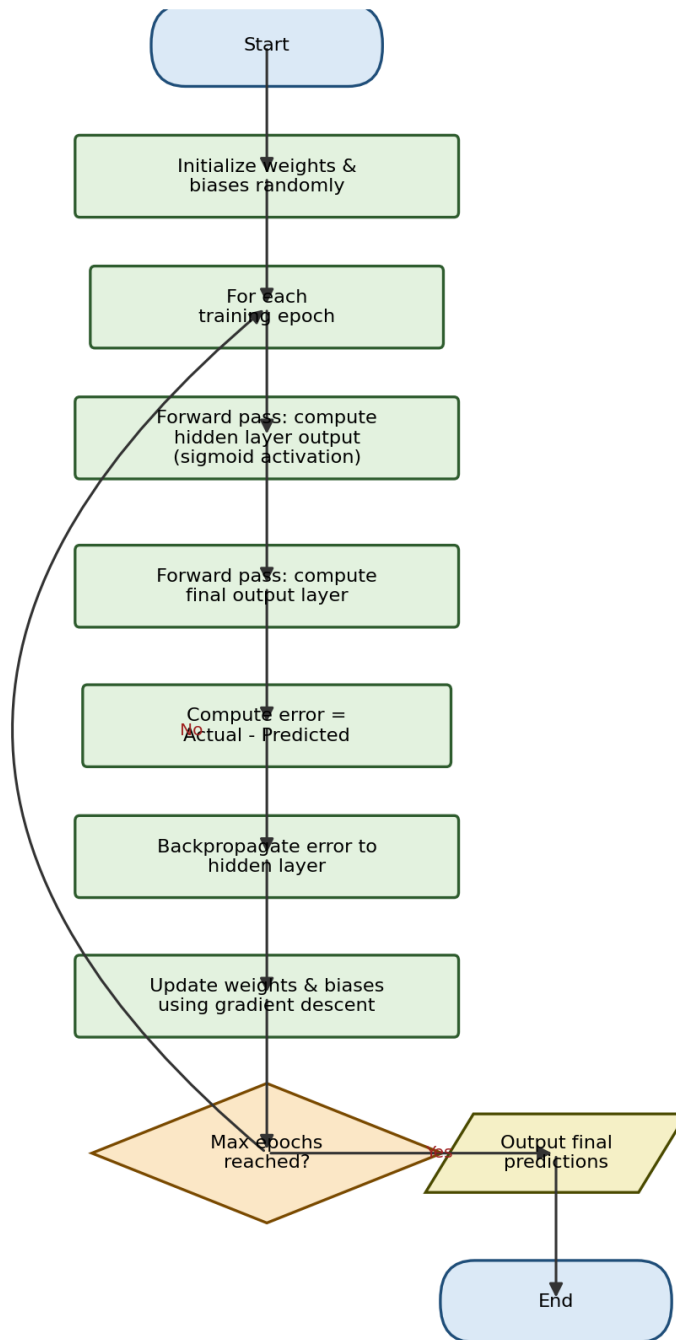
Aim

To write a Python program to implement a feed forward neural network with error back-propagation, using NumPy, to learn the XOR function.

Algorithm

1. Start.
2. Define the training inputs X (the four XOR input combinations) and the target outputs y.
3. Randomly initialize the weights and biases for the input-to-hidden layer ($W1$, $b1$) and the hidden-to-output layer ($W2$, $b2$).
4. Repeat the following steps for a fixed number of epochs:
 5. a) Forward Pass: compute the hidden layer input ($X.W1 + b1$) and apply the sigmoid activation function to obtain the hidden layer output.
 6. b) Compute the final layer input ($\text{hidden_output}.W2 + b2$) and apply the sigmoid activation function to obtain the predicted output.
 7. c) Compute the error as the difference between the actual output y and the predicted output.
 8. d) Backward Pass: compute the gradient at the output layer using the error and the derivative of the sigmoid function.
 9. e) Propagate the error backwards to compute the gradient at the hidden layer.
 10. f) Update the weights and biases of both layers using the computed gradients and the learning rate (Gradient Descent).
11. After training is complete, compute the final predicted output for each input pattern using the trained weights.
12. Round the predicted outputs and compare them with the actual target values.
13. Display the final predictions.
14. Stop.

Flowchart

Flowchart: Feed Forward Neural Network

Program

```

"""
Program 15: Feed Forward Neural Network
A simple feed forward neural network (with backpropagation training)
implemented using NumPy to learn the XOR function
"""

import numpy as np

np.random.seed(1)

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# XOR truth table as training data
X = np.array([[0, 0],
              [0, 1],
              [1, 0],
              [1, 1]])

y = np.array([[0], [1], [1], [0]])

input_neurons = 2
hidden_neurons = 4
output_neurons = 1

# Randomly initialize weights and biases
W1 = np.random.uniform(size=(input_neurons, hidden_neurons))
b1 = np.random.uniform(size=(1, hidden_neurons))
W2 = np.random.uniform(size=(hidden_neurons, output_neurons))
b2 = np.random.uniform(size=(1, output_neurons))

learning_rate = 0.5
epochs = 10000

```

```

for epoch in range(epochs):
    # ---- Forward pass ----
    hidden_input = np.dot(X, W1) + b1
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W2) + b2
    predicted_output = sigmoid(final_input)

    # ---- Backward pass (error backpropagation) ----
    error = y - predicted_output
    d_predicted_output = error * sigmoid_derivative(predicted_output)

    error_hidden = d_predicted_output.dot(W2.T)
    d_hidden_output = error_hidden * sigmoid_derivative(hidden_output)

    # ---- Update weights and biases ----
    W2 += hidden_output.T.dot(d_predicted_output) * learning_rate
    b2 += np.sum(d_predicted_output, axis=0, keepdims=True) * learning_rate
    W1 += X.T.dot(d_hidden_output) * learning_rate
    b1 += np.sum(d_hidden_output, axis=0, keepdims=True) * learning_rate

    if epoch % 2000 == 0:
        loss = np.mean(np.square(error))
        print(f'Epoch {epoch:5d} Loss: {loss:.6f}')

print("\nTraining complete.\n")
print("Final predicted output after training:")
for i in range(len(X)):
    print(f'Input: {X[i]} -> Predicted: {predicted_output[i][0]:.4f} '
          f'Rounded: {round(predicted_output[i][0])} Actual: {y[i][0]}')

```

Sample Input

XOR truth table used as training data:

(0,0)->0 (0,1)->1 (1,0)->1 (1,1)->0

Trained for 10,000 epochs with learning rate = 0.5

Sample Output

```
Epoch 0 Loss: 0.367371
Epoch 2000 Loss: 0.005436
Epoch 4000 Loss: 0.000965
Epoch 6000 Loss: 0.000475
Epoch 8000 Loss: 0.000307
```

Training complete.

Final predicted output after training:

```
Input: [0 0] -> Predicted: 0.0123 Rounded: 0 Actual: 0
Input: [0 1] -> Predicted: 0.9881 Rounded: 1 Actual: 1
Input: [1 0] -> Predicted: 0.9823 Rounded: 1 Actual: 1
Input: [1 1] -> Predicted: 0.0172 Rounded: 0 Actual: 0
```

The training loss steadily decreases from about 0.367 to about 0.0003. After training, the network correctly predicts the XOR outputs: 0, 1, 1, 0 (rounded), matching the actual values.

Sample Screenshot (Program running in VS Code)

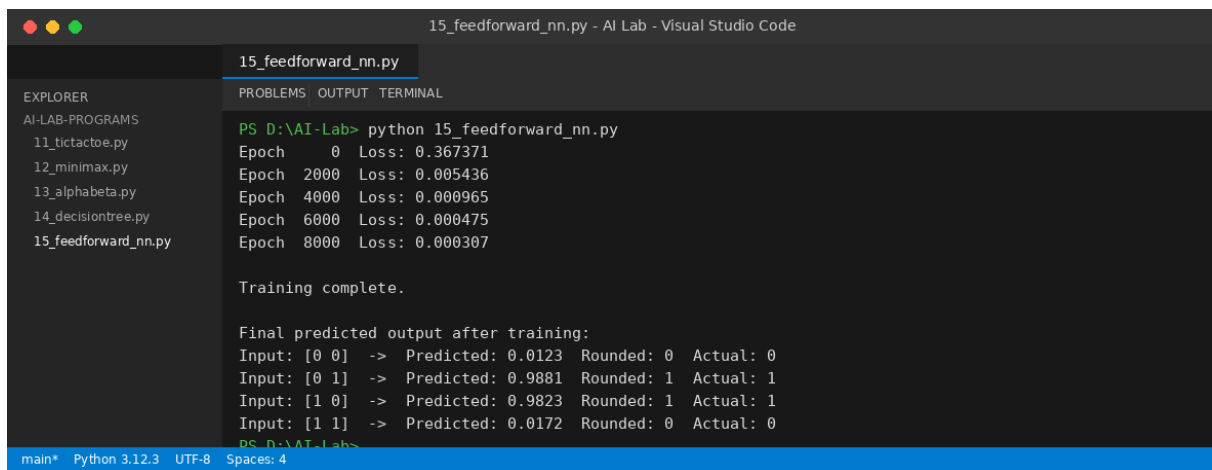
```

1  """
2  Program 15: Feed Forward Neural Network
3  A simple feed forward neural network (with backpropagation training)
4  implemented using NumPy to learn the XOR function
5  """
6
7  import numpy as np
8
9  np.random.seed(1)
10
11 def sigmoid(x):
12     return 1 / (1 + np.exp(-x))
13
14 def sigmoid_derivative(x):
15     return x * (1 - x)
16
17 # XOR truth table as training data
18 X = np.array([[0, 0],
19               [0, 1],
20               [1, 0],
21               [1, 1]])
22
23 y = np.array([[0], [1], [1], [0]])
24
25 input_neurons = 2
26 hidden_neurons = 4
27 output_neurons = 1
28
29 # Randomly initialize weights and biases
30 W1 = np.random.uniform(size=(input_neurons, hidden_neurons))
31 b1 = np.random.uniform(size=(1, hidden_neurons))
32 W2 = np.random.uniform(size=(hidden_neurons, output_neurons))
33 b2 = np.random.uniform(size=(1, output_neurons))
34
35 learning_rate = 0.5
36 epochs = 10000
37
38 for epoch in range(epochs):
39     # ---- Forward pass ----
40     hidden_input = np.dot(X, W1) + b1
41     hidden_output = sigmoid(hidden_input)
42
43     final_input = np.dot(hidden_output, W2) + b2
44     predicted_output = sigmoid(final_input)
45
46     # ---- Backward pass (error backpropagation) ----
47     error = y - predicted_output
48     d_predicted_output = error * sigmoid_derivative(predicted_output)
49
50     error_hidden = d_predicted_output.dot(W2.T)
51     d_hidden_output = error_hidden * sigmoid_derivative(hidden_output)
52
53     # ---- Update weights and biases ----
54     W2 += hidden_output.T.dot(d_predicted_output) * learning_rate
55     b2 += np.sum(d_predicted_output, axis=0, keepdims=True) * learning_rate
56     W1 += X.T.dot(d_hidden_output) * learning_rate
57     b1 += np.sum(d_hidden_output, axis=0, keepdims=True) * learning_rate
58
59     if epoch % 2000 == 0:
60         loss = np.mean(np.square(error))
61         print(f"Epoch {epoch:5d} Loss: {loss:.6f}")
62
63 print("\nTraining complete.\n")
64 print("Final predicted output after training:")
65 for i in range(len(X)):
66     print(f"Input: {X[i]} -> Predicted: {predicted_output[i][0]:.4f} "
67           f"Rounded: {round(predicted_output[i][0])} Actual: {y[i][0]}")

```

main* Python 3.12.3 UTF-8 Spaces: 4 Ln 1, Col 1

Fig 15.1 - Editor view of 15_feedforward_nn.py in Visual Studio Code



```
15_feedforward_nn.py - AI Lab - Visual Studio Code
15_feedforward_nn.py
EXPLORER
AI-LAB-PROGRAMS
11_tictactoe.py
12_minimax.py
13_alphabeta.py
14_decisiontree.py
15_feedforward_nn.py
PROBLEMS OUTPUT TERMINAL
PS D:\AI-Lab> python 15_feedforward_nn.py
Epoch   0  Loss: 0.367371
Epoch 2000  Loss: 0.005436
Epoch 4000  Loss: 0.000965
Epoch 6000  Loss: 0.000475
Epoch 8000  Loss: 0.000307

Training complete.

Final predicted output after training:
Input: [0 0] -> Predicted: 0.0123 Rounded: 0 Actual: 0
Input: [0 1] -> Predicted: 0.9881 Rounded: 1 Actual: 1
Input: [1 0] -> Predicted: 0.9823 Rounded: 1 Actual: 1
Input: [1 1] -> Predicted: 0.0172 Rounded: 0 Actual: 0
PS D:\AI-Lab>
```

Fig 15.2 - Integrated terminal showing the program output

Result

Thus the Python program for "Feed Forward Neural Network" was executed successfully and the output was verified.